



**Indian Institute of Information Technology,
Vadodara**

ONLINE ATTENDANCE SYSTEM

A Project Report

Submitted by:

Aditya Kumar	(202452304)
Ved Gupta	(202451166)
Sanskar Sharma	(202451148)
Prayag Kirar	(202451134)
Priyanshu Virani	(202451169)

Course: Software Engineering

Submitted to: Dr. Pooja Mishra

Abstract

This project presents the design and implementation of a web-based Online Attendance Management System developed to simplify and modernize the process of attendance tracking in educational institutions. Traditional attendance methods are often time-consuming, error-prone, and inefficient. The proposed system provides a secure and automated digital solution that enhances accuracy, transparency, and ease of use.

The system is built with a role-based architecture consisting of three main users: Admin, Teacher, and Student. Teachers can create classes and generate unique access codes, which are shared only with authorized students. Students can join classes using these codes and mark their attendance on a daily basis. Teachers have full control over their classes, including the ability to manage students and remove unauthorized participants. The Admin oversees the system by monitoring teachers and their respective classes.

The application utilizes Firebase Authentication to ensure secure login and user management, and Firebase Realtime Database for instant synchronization of attendance data. This enables real-time updates, scalability, and efficient data handling with minimal latency.

Key features of the system include role-based access control, class creation and management, secure class joining via unique codes, real-time attendance tracking, and an intuitive dashboard interface. This project demonstrates the effective use of modern web technologies and cloud-based backend services to build a reliable and scalable attendance solution.

Acknowledgement

We would like to express our sincere gratitude to our subject teacher and project guide, **Dr. Pooja Mishra**, for her continuous support, valuable guidance, and encouragement throughout the development of this project. Her insights and suggestions greatly helped us in improving the quality of our work.

We also extend our thanks to the Teaching Assistants (TAs) for their assistance, timely feedback, and support during the course of this project.

Finally, we would like to thank our institution for providing us with the necessary resources and environment to successfully complete this project.

Team Members

Signature:

Contents

1	Introduction	5
1.1	Project Overview	5
1.2	Purpose of the Project	5
1.3	Scope of the Project	7
1.3.1	In-Scope Functionalities	7
1.3.2	Out-of-Scope Functionalities	7
2	Requirement Analysis	11
2.1	Project Overview	11
2.2	Purpose	11
2.3	Scope	11
2.4	Functional Requirements	11
2.4.1	User & Authentication Management	12
2.4.2	Attendance Management	12
2.4.3	Attendance Viewing & Sharing	12
2.4.4	Reports & Analytics	12
2.5	Non-Functional Requirements	12
2.6	Constraints and Assumptions	13
2.7	Summary of Requirements	13
3	Design and Architecture	14
3.1	System Structure	14
4	Security Rules on Firebase	14
5	Firebase Realtime Database — Sample Data	16
6	Implementation Details — Realtime Database Module	17
6.1	Module Setup	17
6.2	Save User to RTDB	18
7	Admin-Side Database Functions	19
7.1	adminDatabase	19
7.2	deletingTeacher	19
8	Teacher-Side Database Functions	20
8.1	teacherDatabase	20
8.2	newClassDb	20
8.3	deleteClass	21
8.4	onConnectingToAdmin	21
9	Attendance Functions	22
9.1	onAddTodayAttendanceDatabase	22
9.2	deletionTodaysAttendanceDatabase	23
9.3	markingAttendanceDatabase	23
9.4	listenToAttendance	24
9.5	removingStudentFromClassDatabase	25

10 Student-Side Database Functions	26
10.1 studentDatabase	26
10.2 onNewClassAddDatabase	26
10.3 leavingClassDatabase	27
11 Module Export	27
12 Testing	28
12.1 Whitebox Testing	28
12.2 Objective	28
12.3 Control-Flow Graphs (CFGs) and Testing Tables	28
12.3.1 A. verifyToken()	28
12.3.2 B. saveUser()	31
12.4 Blackbox Testing	34
12.4.1 Introduction	34
12.4.2 Objectives	34
12.4.3 Scope	34
12.4.4 Test Cases - Authentication	34
12.4.5 Test Cases - Student Management	34
12.4.6 Test Cases - Attendance	35
12.4.7 Testing Techniques Used	35
12.4.8 Conclusion	35
13 Functions Covered	36
13.1 A. onNewClassAdd()	36
13.2 B. leavingClass()	38
13.3 C. onNewClassAddDatabase()	40
13.4 D. leavingClassDatabase()	40
14 Conclusion and Future Work	41
14.1 Conclusion	41
14.2 Future Work	42

Introduction

Project Overview

The Attendance Management System is a full-stack web-based application developed to digitize and streamline the traditional process of recording and managing student attendance in educational institutions such as schools and colleges.

In conventional systems, attendance is often recorded manually on paper or basic spreadsheets, which can lead to inefficiencies such as data loss, human errors, difficulty in tracking long-term records, and lack of real-time accessibility. This project addresses these challenges by providing a centralized and automated platform where attendance data can be recorded, stored, processed, and analyzed efficiently.

The system is primarily designed with a teacher-centric architecture, where teachers act as administrators of their respective classes. Each teacher can create and manage multiple classes, and within each class, maintain a structured list of students. A unique identifier (such as a class ID or admin ID) is generated for each class, allowing controlled access and easy sharing.

The core functionality of the system revolves around attendance recording and monitoring. Teachers can mark attendance for students in a simple and user-friendly interface, and the system securely stores this data in a database. Over time, this data is used to generate meaningful insights such as attendance percentages, trends, and summaries, helping both teachers and students track academic participation.

From the student perspective, the system provides visibility into their attendance records, enabling them to stay informed about their attendance status and take corrective actions if needed. This promotes transparency and accountability in the learning environment.

The application follows a modular architecture, typically consisting of:

- A **frontend layer** (built using technologies like HTML, CSS, and JavaScript/React) for user interaction
- A **backend layer** (using Node.js and Express) to handle business logic and API requests
- A **database layer** (such as MongoDB) to store user, class, and attendance data

Overall, the Attendance Management System aims to provide a reliable, efficient, and user-friendly solution that reduces administrative workload, improves data accuracy, and enhances the overall academic management process through digital transformation.

Purpose of the Project

The primary purpose of the Attendance Management System is to design and develop a scalable, secure, and efficient web-based platform that automates the process of recording,

managing, and analyzing student attendance data.

From a technical perspective, the system aims to replace traditional manual attendance methods with a digitized solution by leveraging modern web technologies and structured database systems. The project focuses on implementing a full-stack architecture that ensures seamless communication between the user interface, server-side logic, and persistent data storage.

The system is designed to achieve the following technical objectives:

- **Automation of Attendance Recording:** Implement mechanisms to digitally capture and store attendance data, minimizing human errors and redundancy associated with manual processes.
- **Efficient Data Management:** Design a normalized database schema to store entities such as teachers, students, classes, and attendance records, ensuring data consistency, integrity, and optimized query performance.
- **RESTful API Design:** Build well-structured RESTful APIs to handle client-server communication, enabling modular development and easy integration with frontend interfaces.
- **Scalability and Modularity:** Apply software engineering principles such as separation of concerns (routes, controllers, services) to ensure the system can scale and be extended with additional features in the future.
- **Real-time Data Access and Visualization:** Provide mechanisms for retrieving and presenting attendance statistics, such as percentage calculations and summaries, to support data-driven decision-making.
- **Error Handling and Validation:** Implement robust input validation and exception handling to maintain system reliability and prevent invalid or inconsistent data entries.
- **User-Centric Interface Design:** Ensure that the frontend interface is intuitive and responsive, allowing users (teachers and students) to interact efficiently with the system.

Additionally, the project serves an academic purpose by providing practical exposure to full-stack development, database design, API architecture, and secure software development practices. It enables the application of theoretical concepts from Software Engineering, Database Management Systems, and Web Development in a real-world problem domain.

Scope of the Project

The scope of the Attendance Management System defines the functional boundaries and capabilities of the application, as well as the limitations beyond which the system does not operate.

1.3.1 In-Scope Functionalities

The system is designed to support the following core features:

- **User Authentication and Authorization:** Secure login and registration functionality for teachers, ensuring controlled access to system resources.
- **Class Management:** Teachers can create, update, and delete classes. Each class is associated with a unique identifier to manage and access class-specific data.
- **Student Management:** Functionality to add, remove, and search students within a class. Each student record includes relevant attributes such as name, roll number, and associated class.
- **Attendance Recording:** Teachers can mark attendance for students on a daily basis. The system stores attendance records with timestamps for future reference.
- **Attendance Retrieval:** Users can view attendance history for individual students or entire classes through structured queries.
- **Statistics and Reporting:** The system computes attendance metrics such as percentage, total /absent counts, and summary reports.
- **RESTful API Integration:** Backend services expose endpoints for all major operations, enabling smooth communication between frontend and backend layers.
- **Responsive User Interface:** A web-based interface that adapts to different screen sizes and provides an intuitive user experience.

1.3.2 Out-of-Scope Functionalities

The following features are not included in the current implementation:

- Biometric or facial recognition-based attendance systems
- Integration with third-party institutional ERP systems
- Native mobile application development (Android/iOS)
- Offline mode or local data synchronization
- Advanced analytics using machine learning or AI techniques

The scope is intentionally limited to ensure focused development, maintain system simplicity, and allow future enhancements without architectural changes.

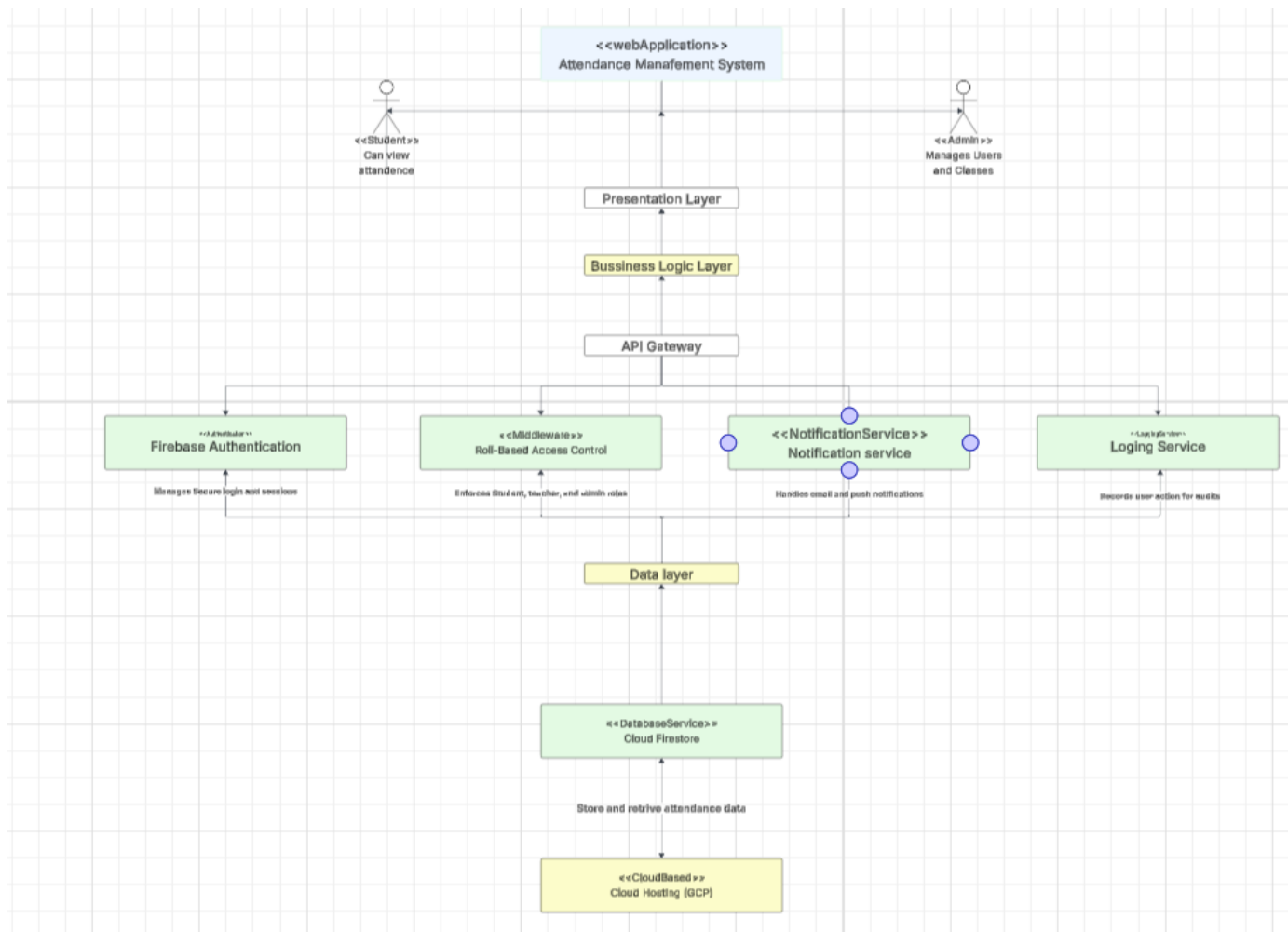


Figure 1: System Architecture Diagram

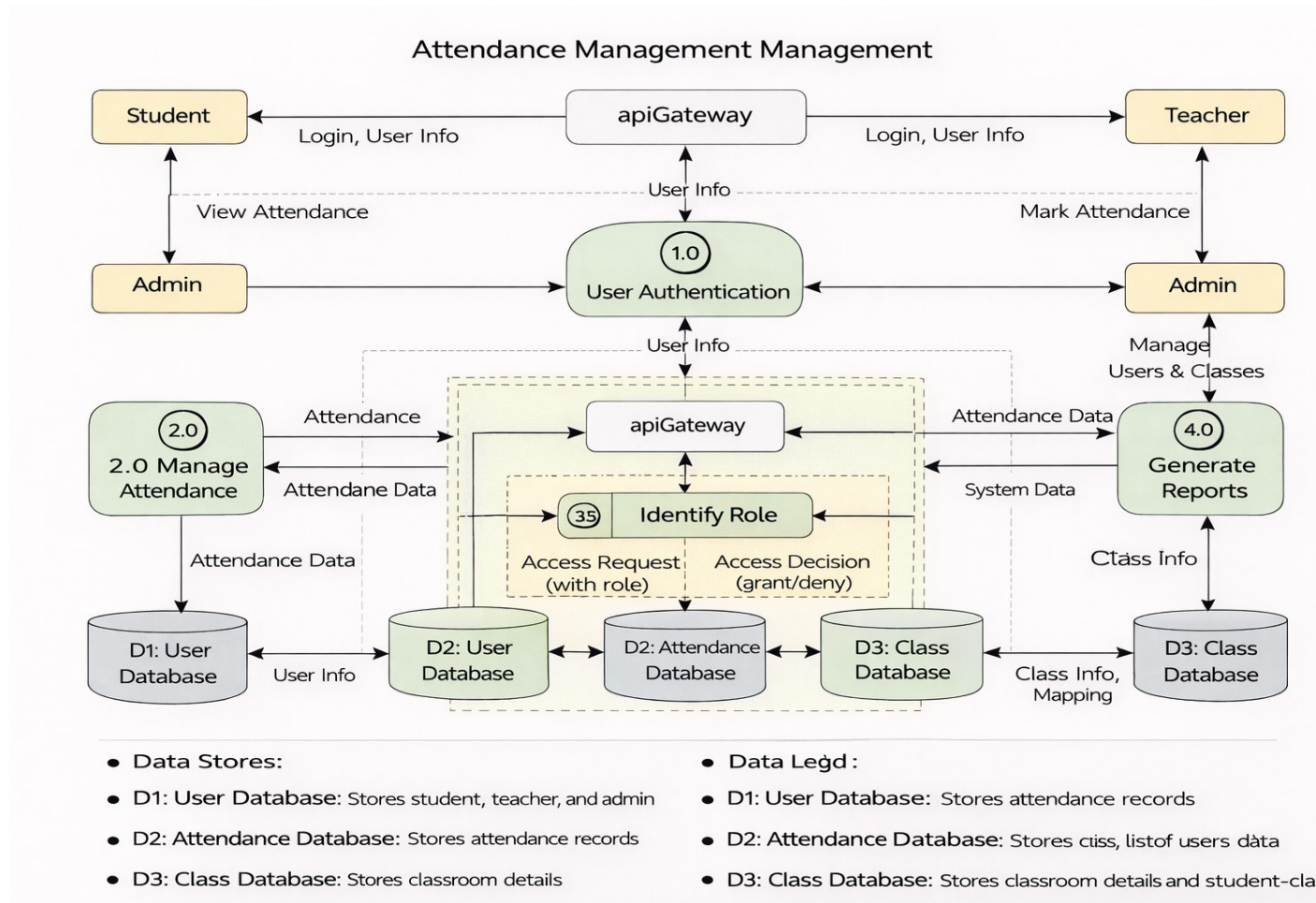


Figure 2: Data flow Diagram

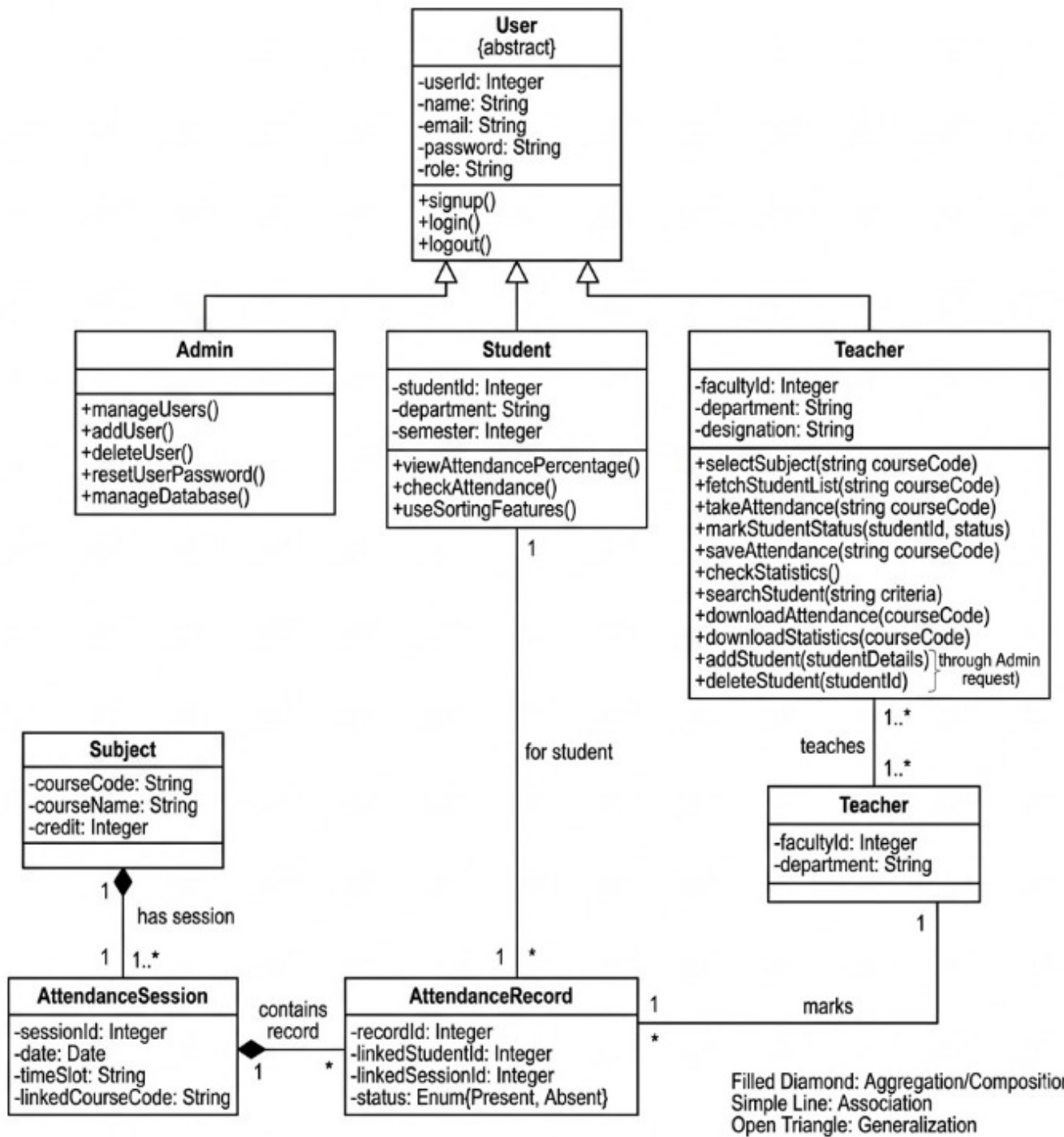


Figure 3: UML class Diagram

Requirement Analysis

The Attendance Management System is a web-based application designed to digitize and simplify the process of recording, storing, viewing, and reporting student attendance in educational institutions. Its main purpose is to reduce manual work, minimize errors, improve transparency, and provide a centralized and secure platform for students, faculty, and administrators. The system is intended to be cloud-based and scalable, with real-time data handling and role-based access control.

Project Overview

The project provides a digital solution for attendance management where:

- **Students** can view their attendance records in read-only mode.
- **Faculty** can create attendance sessions, mark attendance, modify records within a limited time window, and generate reports.
- **Administrators** can manage user accounts, monitor attendance across classes, and oversee system data.

Purpose

The purpose of this system is to replace traditional manual attendance methods with an efficient online platform. It helps in:

- reducing paperwork,
- avoiding duplicate and incorrect attendance entries,
- providing secure and instant access to attendance data,
- enabling better monitoring of student attendance performance.

Scope

The scope of the system includes user authentication, attendance marking, attendance viewing, report generation, and administrative control. It supports multiple user roles and is designed for use in colleges and universities. The system also supports cloud-based storage, real-time synchronization, and browser-based access on desktop and mobile devices.

Functional Requirements

The system must support three user roles:

- Student
- Faculty
- Administrator

It must provide role-based access control to ensure that users can only access the functions allowed for their role.

2.4.1 User & Authentication Management

- Users shall log in using valid credentials.
- The system shall authenticate users based on assigned roles.
- Unauthorized access to attendance data shall be prevented.
- Administrators shall be able to create, update, or deactivate user accounts.

2.4.2 Attendance Management

- Faculty shall be able to create attendance sessions for a specific class and date.
- Faculty shall mark students as Present or Absent.
- Attendance records shall be stored securely in cloud storage.
- Duplicate attendance entries for the same student on the same date shall not be allowed.
- Faculty shall be allowed to modify attendance records within a defined time window.

2.4.3 Attendance Viewing & Sharing

- Students shall be able to view their attendance records in read-only mode.
- Faculty shall be able to share attendance records with authorized users.
- Administrators shall be able to view attendance across all classes.
- The system shall generate attendance summaries based on date, subject, or student.

2.4.4 Reports & Analytics

- The system shall generate attendance reports in tabular format.
- It shall calculate attendance percentage for each student.
- Faculty and administrators shall be able to filter reports by date or subject.
- Reports shall be downloadable or exportable.

Non-Functional Requirements

- Authentication should be completed within 2 seconds.
- Attendance marking and submission should work in real time.
- Data retrieval should not take more than 3 seconds under normal network conditions.
- The system should support concurrent users without data inconsistency.
- Client-side memory usage should remain minimal.
- Communication should be secure through HTTPS.
- The application should comply with institutional privacy and security policies.

Constraints and Assumptions

- The system will be developed as a web application only.
- It must run on modern web browsers.
- Firebase Authentication will be used for login and session management.
- Firebase Realtime Database or Firestore will be used for data storage and synchronization.
- The system will follow role-based access control.
- The project should be implementable within limited academic time and budget using free or low-cost tools.

Summary of Requirements

In summary, the system is required to provide secure login, efficient attendance management, read-only student access, administrative control, and report generation with analytics. It should be reliable, scalable, user-friendly, and suitable for educational institutions that need a modern digital attendance solution.

Design and Architecture

System Structure

The project follows a standard React application layout with a dedicated `src/` directory for all source code, separated into components, pages, assets, and configuration.

```

1 Attendance-Management-System/
2 |
3 +-- node_modules/           # Installed dependencies
4 +-- public/                 # Static files
5 |
6 +-- src/                    # Main source code
7 | |
8 | | +-- assets/             # Images, icons, and static resources
9 | | |
10 | | +-- Components/        # Reusable UI components
11 | | |
12 | | +-- Pages/             # Main application pages
13 | | |   +-- AdminDashboard.jsx
14 | | |   +-- AdminLoginForm.jsx
15 | | |   +-- ClassAttendance.jsx
16 | | |   +-- SignupForm.jsx
17 | | |   +-- StudentDashboard.jsx
18 | | |   +-- TeacherDashboard.jsx
19 | | |   +-- UserRole.jsx
20 | | |
21 | | +-- App.jsx             # Root component
22 | | +-- App.css             # Global styles for App
23 | | +-- index.css           # Base styles
24 | | +-- main.jsx            # Entry point of React app
25 | |
26 +-- .gitignore              # Git ignored files
27 +-- eslint.config.js        # Linting configuration
28 +-- index.html              # Main HTML file

```

Listing 1: Project Directory Tree

Security Rules on Firebase

The following Firebase Realtime Database security rules govern read and write access for each role (admin, teacher, student) and each data node (users, class, attendance).

```

1 {
2   "rules": {
3
4     "users": {
5       "$uid": {
6         ".read": "auth != null",
7         ".write": "auth.uid === $uid"
8       }
9     }
10  }

```

```

9      },
10
11      "student": {
12          "$uid": {
13              ".read": "auth != null",
14              ".write": "auth.uid === $uid"
15          }
16      },
17
18      "teacher": {
19          "$teacherId": {
20              ".read": "auth != null",
21              ".write": "auth.uid === $teacherId ||
22                  (data.child('admin').val() === auth.uid) ||
23                  (!newData.exists() && data.child('admin').val() ===
24                      auth.uid)"
25          }
26      },
27
28      "admin": {
29          "$uid": {
30              ".read": "auth != null",
31              ".write": "auth.uid === $uid"
32          }
33      },
34
35      "class": {
36          "$classId": {
37              ".read": "auth != null",
38              ".write":
39                  "root.child('users').child(auth.uid).child('role').val
40                      ()
41                      === 'teacher' ||
42                      root.child('users').child(auth.uid).child('role').val
43                      ()
44                      === 'admin'",
45
46          "student": {
47              "$studentUid": {
48                  ".write": "auth.uid === $studentUid",
49                  ".read": "auth != null"
50              }
51          },
52
53          "attendance": {
54              ".write":
55                  "root.child('users').child(auth.uid).child('role').
56                      val()
57                      === 'teacher'",
58              ".read": "auth != null"
59          }
60      }
61  }

```



```

56   }
57   }
58 }
59 }

```

Listing 2: Firebase Security Rules

Firebase Realtime Database — Sample Data

```

1 {
2   "admin": {
3     "QYJ1r66aeoRMUfOMMpFTWvjburq2": {
4       "adminName": "lsfjg",
5       "instituteName": "akjskg",
6       "role": "admin",
7       "teacher": {
8         "QYJ1r66aeoRMUfOMMpFTWvjburq2": true,
9         "uS2eXaciT9cCb7U7RRWe1NIq8Dk1": true
10      }
11    },
12    "uS2eXaciT9cCb7U7RRWe1NIq8Dk1": {
13      "adminName": "Bhupendra Patel",
14      "instituteName": "Indian Institute of Information
15        Technology",
16      "role": "admin",
17      "teacher": {
18        "QYJ1r66aeoRMUfOMMpFTWvjburq2": true,
19        "uS2eXaciT9cCb7U7RRWe1NIq8Dk1": true
20      }
21    },
22  },
23  "class": {
24    "Dt-rQ-JF7kx8X11": {
25      "attendance": {
26        "1-4-2026": { "uS2eXaciT9cCb7U7RRWe1NIq8Dk1": false },
27        "2-4-2026": { "uS2eXaciT9cCb7U7RRWe1NIq8Dk1": false },
28        "3-4-2026": { "uS2eXaciT9cCb7U7RRWe1NIq8Dk1": true },
29        "4-4-2026": { "uS2eXaciT9cCb7U7RRWe1NIq8Dk1": true },
30        "6-4-2026": { "uS2eXaciT9cCb7U7RRWe1NIq8Dk1": true },
31        "7-4-2026": { "uS2eXaciT9cCb7U7RRWe1NIq8Dk1": false }
32      },
33      "classId": "Dt-rQ-JF7kx8X11",
34      "courseName": "Software Engineering",
35      "requiredattendance": "90",
36      "student": {
37        "uS2eXaciT9cCb7U7RRWe1NIq8Dk1": {
38          "totalDays": 9,
39          "totalPresent": 6
40        }
41      },

```

```

42     "teacherId": "uS2eXaciT9cCb7U7RRWe1NIq8Dk1"
43   }
44 },
45
46 "student": {
47   "uS2eXaciT9cCb7U7RRWe1NIq8Dk1": {
48     "class": {
49       "_ck37EQU5gBzqzo": true,
50       "nMaxwu9q3astw0v": true,
51       "sDIAZF_jIyJZNKs": true
52     },
53     "id": "202452304",
54     "name": "Aditya Kumar",
55     "role": "student"
56   }
57 },
58
59 "teacher": {
60   "QYJ1r66aeoRMUfOMMpFTWvjburq2": {
61     "admin": "uS2eXaciT9cCb7U7RRWe1NIq8Dk1",
62     "class": {
63       "0YcYmq7bXm3WajZ": true,
64       "Cvgm6FeQmreWz0h": true,
65       "lHbY_m5wNfzc3kf": true
66     },
67     "id": "789",
68     "name": "ved",
69     "role": "teacher"
70   }
71 },
72
73 "users": {
74   "uS2eXaciT9cCb7U7RRWe1NIq8Dk1": {
75     "email": "r2aditya67@gmail.com",
76     "id": "uS2eXaciT9cCb7U7RRWe1NIq8Dk1",
77     "name": "Aditya Kumar",
78     "role": "Student",
79     "session": "2025-2029"
80   }
81 }
82 }

```

Listing 3: Firebase RTDB Sample Data

Implementation Details — Realtime Database Module

Module Setup

```
1 import { db } from '../config/firebaseAdmin.js'
```

Listing 4: Firebase DB Import

Save User to RTDB

Persists a newly authenticated user's profile and delegates to the appropriate role-specific database function (`adminDatabase`, `teacherDatabase`, or `studentDatabase`).

```
1  async function saveUserToRTDB(user, role, personalInfo) {
2    const userRef = db.ref("users/" + user.uid);
3    const userData = {
4      id: user.uid,
5      role: role,
6      name: user.name,
7      email: user.email,
8      picture: user.picture,
9      session: "2025-2029"
10   };
11   await userRef.set(userData);
12
13   if (role == 'admin') {
14     const adminRef = db.ref('admin/${user.uid}');
15     try {
16       const snapshot = await adminRef.once('value');
17       if (snapshot.exists()) return false;
18       else return await adminDatabase(user, personalInfo);
19     } catch (error) {
20       console.error(error);
21       return false;
22     }
23   } else if (role == 'Teacher') {
24     const teacherRef = db.ref('teacher/${user.uid}');
25     try {
26       const snapshot = await teacherRef.once('value');
27       if (snapshot.exists()) return false;
28       else return await teacherDatabase(user, personalInfo)
29       ;
30     } catch (error) {
31       console.error(error);
32       return false;
33     }
34   } else if (role == 'Student') {
35     const studentRef = db.ref('student/${user.uid}');
36     try {
37       const snapshot = await studentRef.once('value');
38       if (snapshot.exists()) return false;
39       else return await studentDatabase(user, personalInfo)
40       ;
41     } catch (error) {
42       console.error(error);
43       return false;
44     }
45   }
```

```
42     }  
43   }  
44 }
```

Listing 5: saveUserToRTDB

Admin-Side Database Functions

adminDatabase

```
1  async function adminDatabase(user, personalInfo) {  
2    try {  
3      const adminRef = db.ref('admin/${user.uid}');  
4      const adminData = {  
5        instituteName: personalInfo.instituteName,  
6        adminName: personalInfo.adminName,  
7        role: 'admin',  
8        teacher: {}  
9      };  
10     await adminRef.set(adminData);  
11     return true;  
12   } catch (error) {  
13     console.error('adminDatabase ${error}');  
14     return false;  
15   }  
16 }
```

Listing 6: adminDatabase

deletingTeacher

Atomically removes a teacher from both the `admin` and `teacher` nodes using a multi-path update.

```
1  async function deletingTeacher(user, teacherId) {  
2    try {  
3      const updates = {};  
4      updates['admin/${user.uid}/teacher/${teacherId}'] = null;  
5      updates['teacher/${teacherId}'] = null;  
6      await db.ref().update(updates);  
7      console.log("Successfully deleted teacher.");  
8      return true;  
9    } catch (error) {  
10     console.error("Delete failed:", error);  
11     return false;  
12   }  
13 }
```

Listing 7: deletingTeacher

Teacher-Side Database Functions

teacherDatabase

```
1 async function teacherDatabase(user, personalInfo) {
2   try {
3     const teacherRef = db.ref('teacher/${user.uid}');
4     const teacherData = {
5       id: personalInfo.id,
6       name: personalInfo.name,
7       role: 'teacher',
8       admin: 1234,
9       class: {}
10    };
11    await teacherRef.set(teacherData);
12    return true;
13  } catch (error) {
14    console.error('teacherDatabase ${error}');
15    return false;
16  }
17 }
```

Listing 8: teacherDatabase

newClassDb

Creates a new class node and registers it under the owning teacher.

```
1 async function newClassDb(user, courseName, requiredattendance,
2                             randomClassId, teacherUid) {
3   try {
4     const targetTeacherUid = teacherUid || user.uid;
5     const classRef = db.ref('class/${randomClassId}');
6     const classData = {
7       classId: randomClassId,
8       teacherId: targetTeacherUid,
9       courseName: courseName,
10      requiredattendance: requiredattendance,
11      student: {},
12      attendance: {}
13    };
14    await classRef.set(classData);
15    const teacherClassRef = db.ref('teacher/${
16      targetTeacherUid}/class');
17    await teacherClassRef.update({ [randomClassId]: true });
18    return true;
19  } catch (error) {
20    console.error('newClassDb ${error}');
21    return false;
22  }
23 }
```

Listing 9: newClassDb

deleteClass

Removes a class and cleans up references in the teacher node and every enrolled student's profile.

```
1 async function deleteClass(user, classId, teacherUid) {
2   try {
3     const targetTeacherUid = teacherUid || user.uid;
4     const classRef = db.ref('class/${classId}');
5     const snapshot = await classRef.child('student').once('
      value');
6     const studentList = snapshot.val() || {};
7
8     const updates = {};
9     updates['class/${classId}'] = null;
10    updates['teacher/${targetTeacherUid}/class/${classId}'] =
      null;
11
12    Object.keys(studentList).forEach((studentUid) => {
13      updates['student/${studentUid}/class/${classId}'] =
        null;
14    });
15    await db.ref().update(updates);
16    return true;
17  } catch (error) {
18    console.error('deleteClass error: ${error.message}');
19    return false;
20  }
21 }
```

Listing 10: deleteClass

onConnectingToAdmin

Links a teacher to an admin by verifying the admin exists, then performing a concurrent update on both nodes.

```
1 async function onConnectingToAdmin(user, adminId) {
2   try {
3     const adminExistingRef = db.ref('admin/${adminId}');
4     const snapshot = await adminExistingRef.once('value');
5     if (!snapshot.exists()) return false;
6
7     const adminRef = db.ref('admin/${adminId}/teacher');
8     const teacherAdminRef = db.ref('teacher/${user.uid}/admin
      ');
9     await Promise.all([
10      adminRef.update({ [user.uid]: true }),
```

```

11         teacherAdminRef.set(adminId)
12     });
13     return true;
14 } catch (error) {
15     console.error('onConnectingToAdmin ${error}');
16     return false;
17 }
18 }

```

Listing 11: onConnectingToAdmin

Attendance Functions

onAddTodayAttendanceDatabase

Initialises today's attendance for all enrolled students. Each student starts as `false` (absent) and `totalDays` is incremented by one.

```

1  async function onAddTodayAttendanceDatabase(classId) {
2      try {
3          const classRef = db.ref('class/${classId}');
4          const snapshot = await classRef.once("value");
5          if (!snapshot.exists()) return false;
6
7          const classData = snapshot.val();
8          const today = new Date();
9          const dateStr =
10              `${today.getDate()}-${today.getMonth() + 1}-${today.
11                  getFullYear()}`;
12
13          if (classData.attendance && classData.attendance[dateStr]) {
14              return false;
15          }
16
17          const studentListObj = classData.student || {};
18          const updates = {};
19
20          Object.keys(studentListObj).forEach((uid) => {
21              updates['class/${classId}/attendance/${dateStr}/${uid}'] = false;
22              const existingStats =
23                  studentListObj[uid] && typeof studentListObj[uid]
24                      === "object"
25                      ? studentListObj[uid]
26                      : {};
27              const currentTotalDays = Number(existingStats.
28                  totalDays) || 0;
29              updates['class/${classId}/student/${uid}/totalDays'] =
30                  currentTotalDays + 1;

```

```

28         updates['class/${classId}/student/${uid}/totalPresent
29             '] =
30             Number(existingStats.totalPresent) || 0;
31     });
32
33     await db.ref().update(updates);
34     return true;
35 } catch (error) {
36     console.error("onAddTodayAttendanceDatabase", error);
37     return false;
38 }

```

Listing 12: onAddTodayAttendanceDatabase

deletionTodaysAttendanceDatabase

```

1  async function deletionTodaysAttendanceDatabase(classId) {
2      try {
3          const today = new Date();
4          const dateStr =
5              `${today.getDate()}-${today.getMonth() + 1}-${today.
6                  getFullYear()}`;
7          const attendanceRef =
8              db.ref(`class/${classId}/attendance/${dateStr}`);
9          await attendanceRef.remove();
10         return true;
11     } catch (error) {
12         console.log(`${error}`);
13         return false;
14     }
15 }

```

Listing 13: deletionTodaysAttendanceDatabase

markingAttendanceDatabase

Toggles a student's present/absent status for today and adjusts `totalPresent` accordingly, guarding against boundary conditions.

```

1  async function markingAttendanceDatabase(studentId, classId,
2      status) {
3      try {
4          const today = new Date();
5          const dateStr =
6              `${today.getDate()}-${today.getMonth() + 1}-${today.
7                  getFullYear()}`;
8
9          const statusRef =
10             db.ref(`class/${classId}/attendance/${dateStr}/${
11                 studentId}`);

```



```

9      const statsRef =
10        db.ref('class/${classId}/student/${studentId}');
11
12      const [currentSnapshot, statsSnapshot] = await Promise.
13        all([
14          statusRef.once('value'),
15          statsRef.once('value')
16        ]);
17      const previousStatus = currentSnapshot.val();
18      if (previousStatus === status) return true;
19
20      const statsData = statsSnapshot.val();
21      const safeStats =
22        statsData && typeof statsData === "object"
23          ? statsData
24          : { totalDays: 0, totalPresent: 0 };
25
26      let nextTotalPresent = Number(safeStats.totalPresent) ||
27        0;
28
29      if (previousStatus === false && status === true) {
30        nextTotalPresent += 1;
31      } else if (previousStatus === true && status === false) {
32        nextTotalPresent = Math.max(0, nextTotalPresent - 1);
33      } else if (previousStatus == null && status === true) {
34        nextTotalPresent += 1;
35      }
36
37      nextTotalPresent =
38        Math.min(nextTotalPresent, Number(safeStats.totalDays
39          ) || 0);
40
41      await statusRef.set(status);
42      await statsRef.update({
43        totalDays: Number(safeStats.totalDays) || 0,
44        totalPresent: nextTotalPresent,
45      });
46      return true;
47    } catch (error) {
48      console.error('markingAttendanceDatabase error: ${error
49        }');
50      return false;
51    }
52  }

```

Listing 14: markingAttendanceDatabase

listenToAttendance

Real-time listener that logs the present/total count whenever today's attendance changes.

```

1 function listenToAttendance(classId) {
2   const today = new Date();
3   const dateStr =
4     `${today.getDate()}-${today.getMonth() + 1}-${today.
       getFullYear()}`;
5   const attendanceRef =
6     db.ref(`class/${classId}/attendance/${dateStr}`);
7
8   onValue(attendanceRef, (snapshot) => {
9     const data = snapshot.val();
10    if (data) {
11      const presentCount =
12        Object.values(data).filter(s => s === true).
          length;
13      const totalStudents = Object.keys(data).length;
14      console.log(`Present: ${presentCount} / Total: ${
          totalStudents}`);
15    }
16  });
17 }

```

Listing 15: listenToAttendance

removingStudentFromClassDatabase

Removes a student from the class, the student's own class list, and today's attendance record — all concurrently.

```

1 async function removingStudentFromClassDatabase(studentUserId,
  classId) {
2   try {
3     const today = new Date();
4     const dateStr =
5       `${today.getDate()}-${today.getMonth() + 1}-${today.
         getFullYear()}`;
6
7     const classStudentRef =
8       db.ref(`class/${classId}/student/${studentUserId}`);
9     const studentProfileRef =
10       db.ref(`student/${studentUserId}/class/${classId}`);
11     const todaysStudentAttendanceRef =
12       db.ref(`class/${classId}/attendance/${dateStr}/${
         studentUserId}`);
13
14     await Promise.all([
15       classStudentRef.remove(),
16       studentProfileRef.remove(),
17       todaysStudentAttendanceRef.remove()
18     ]);
19     return true;
20   } catch (error) {
21     console.error("removingStudentFromClassDatabase", error);

```

```
22     return false;
23   }
24 }
```

Listing 16: removingStudentFromClassDatabase

Student-Side Database Functions

studentDatabase

```
1  async function studentDatabase(user, personalInfo) {
2    try {
3      const studentRef = db.ref('student/${user.uid}');
4      const studentData = {
5        id: personalInfo.id,
6        name: personalInfo.name,
7        role: 'student',
8        class: {}
9      };
10     await studentRef.set(studentData);
11   } catch (error) {
12     console.error('studentDatabase ${error}');
13     return false;
14   }
15 }
```

Listing 17: studentDatabase

onNewClassAddDatabase

Enrolls a student in a class by writing their initial stats to the class node and registering the class in their own profile.

```
1  async function onNewClassAddDatabase(user, classId) {
2    try {
3      const classRef = db.ref('class/${classId}');
4      const snapshot = await classRef.once('value');
5      if (snapshot.exists()) {
6        const classStudentRef =
7          db.ref('class/${classId}/student/${user.uid}');
8        const studentRef =
9          db.ref('student/${user.uid}/class/${classId}');
10       await Promise.all([
11         classStudentRef.set({ totalPresent: 0, totalDays:
12           0 }),
13         studentRef.set(true)
14       ]);
15       return true;
16     } catch (error) {
17       console.error('onNewClassAddDatabase ${error}');
```

```
18         return false;
19     }
20 }
```

Listing 18: onNewClassAddDatabase

leavingClassDatabase

```
1 async function leavingClassDatabase(user, classId) {
2     try {
3         const studentRef =
4             db.ref('student/${user.uid}/class/${classId}');
5         await studentRef.remove();
6         return true;
7     } catch (error) {
8         console.error('leavingClassDatabase ${error}');
9         return false;
10    }
11 }
```

Listing 19: leavingClassDatabase

Module Export

All database functions are collected into a single DB object and exported as the default export of the module.

```
1 const DB = {
2     leavingClassDatabase,
3     deleteClass,
4     markingAttendanceDatabase,
5     onAddTodayAttendanceDatabase,
6     onConnectingToAdmin,
7     onNewClassAddDatabase,
8     saveUserToRTDB,
9     newClassDb,
10    removingStudentFromClassDatabase,
11    deletionTodaysAttendanceDatabase,
12    deletingTeacher,
13 };
14 export default DB;
```

Listing 20: DB export object

Testing

Whitebox Testing

Objective

To perform white box testing on the backend controller modules of the Online Attendance System by analyzing internal code structure, control flow, and execution paths. The goal is to ensure correctness of logic, proper handling of conditions, and full code coverage.

Control-Flow Graphs (CFGs) and Testing Tables

12.3.1 A. `verifyToken()`

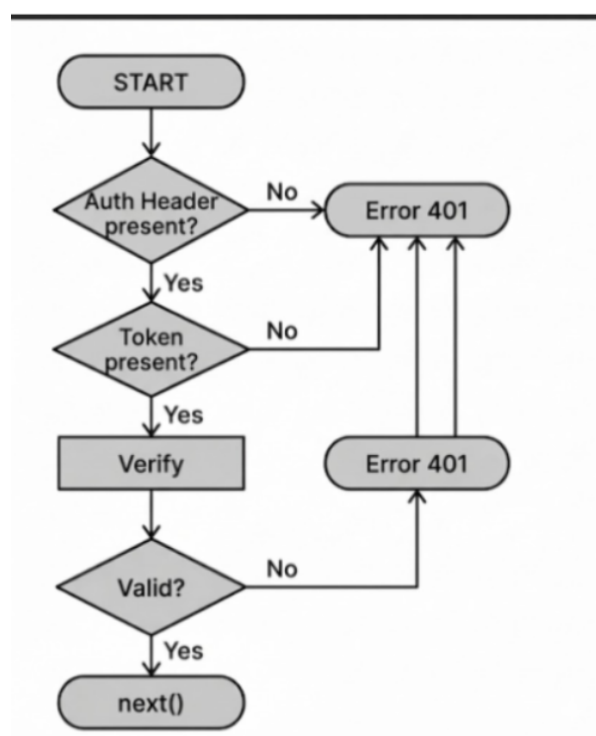


Figure 4: Image 1

Cyclomatic Complexity

- $M = DecisionPoints + 1$
- Decision Points:
 - `if (!authHeader)`
 - `if (!token)`
 - `try-catch` (considered decision)
- Total = 3
- $M = 3 + 1 = 4$

Test Cases

We have to test 4 independent paths.

LXII

TC	Input	Expected Output	Path
TC1	No header	401 error	Path 1
TC2	Header but no token	401 error	Path 2
TC3	Invalid token	401 error	Path 3
TC4	Valid token	next() call	Path 4

TC1: No Header

Put Headers empty in Thunder Client.

Expected:

```
{
  "error": "No authorization header"
}
```

TC2: Header but No Token**Headers:**

```
{
  "Authorization": "Bearer "
}
```

Expected:

```
{
  "error": "No token provided"
}
```

TC3: Invalid Token**Headers:**

```
{
  "Authorization": "Bearer fake123"
}
```

Expected:

```
{
  "error": "Invalid token"
}
```

TC4: Valid Token**Headers:**

```
{  
  "Authorization": "Bearer VALID_FIREBASE_TOKEN"  
}
```

Expected:

```
{  
  "data": true,  
  "message": "successfully connected"  
}
```

12.3.2 B. saveUser()

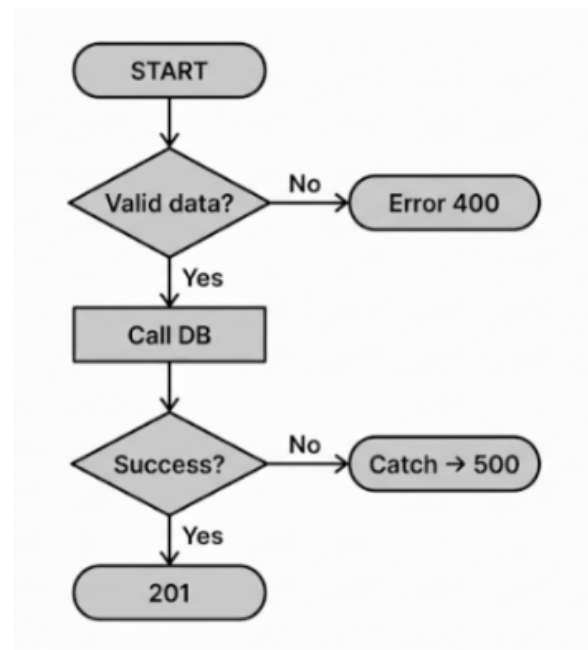


Figure 5: Image 2

Cyclomatic Complexity

- Decision Points:
 - if (!req.user || !role || !personalInfo)
 - try-catch
- Total = 2
- $M = 2 + 1 = 3$

We have to test 3 paths.

Test Cases

1X1

TC Input Expected Output

TC1 Missing data 400 error

TC2 Valid data + DB success 201 success

TC3 DB failure 500 error

TC5: Missing All Fields

Input:


```
{}
```

Expected:

```
{  
  "error": "there some error occurred"  
}
```

TC6: Missing personalInfo**Input:**

```
{  
  "role": "admin"  
}
```

Expected:

```
{  
  "error": "there some error occurred"  
}
```

TC7: Missing role**Input:**

```
{  
  "personalInfo": {  
    "adminName": "Rahul",  
    "instituteName": "ABC College"  
  }  
}
```

Expected:

```
{  
  "error": "there some error occurred"  
}
```

TC8: Valid Admin Data**Input:**

```
{  
  "role": "admin",  
  "personalInfo": {  
    "adminName": "Rahul",  
    "instituteName": "ABC College"  
  }  
}
```

Expected:

```
{  
  "data": true,  
  "message": "successfully connected with the database"  
}
```

TC9: Valid Teacher Data**Input:**

```
{
  "role": "Teacher",
  "personalInfo": {
    "teacherName": "Amit",
    "subject": "Math"
  }
}
```

Expected:

```
{
  "data": true,
  "message": "successfully connected with the database"
}
```

TC10: Valid Student Data**Input:**

```
{
  "role": "Student",
  "personalInfo": {
    "studentName": "Ravi",
    "branch": "IT"
  }
}
```

Expected:

```
{
  "data": true,
  "message": "successfully connected"
}
```

TC11: Invalid Role**Input:**

```
{
  "role": "guest",
  "personalInfo": {
    "name": "Test"
  }
}
```

Expected:

```
{
  "data": null
}
```

Blackbox Testing

12.4.1 Introduction

Blackbox testing is a method where the internal implementation is hidden and testing is based only on inputs and outputs. This document validates the Online Attendance System functionalities. [?]

12.4.2 Objectives

- Validate system functionality
- Ensure correct outputs
- Detect errors
- Verify system reliability

12.4.3 Scope

Modules tested include:

- Authentication
- Student Management
- Attendance
- Statistics
- Dashboard

12.4.4 Test Cases - Authentication

ID	Description	Input	Expected Output	Result
TC01	Valid Login	Correct credentials	Login success	Pass
TC02	Invalid Password	Wrong password	Error message	Pass
TC03	Empty Fields	No input	Validation error	Pass
TC04	Invalid User	Random Username	User not exists	Pass

12.4.5 Test Cases - Student Management

ID	Description	Input	Expected Output	Result
TC04	Add student	Class Exists	Student added	Pass
TC05	Add student	Class doesn't exist	Error shown	Pass
TC06	Remove student	Student removed	Record deleted	Pass
TC07	Add new student	Valid input	Appears in list	Pass
TC08	Dynamic classes	Real-time update	Classes updated	Pass

12.4.6 Test Cases - Attendance

ID	Description	Input	Expected Output	Result
TC09	Mark attendance	Student marked	Saved in DB	Pass
TC10	Default attendance	No action	Marked absent	Pass
TC11	Analytics	Class data	Percentage shown	Pass
TC12	Modify attendance	Same day edit	Updated successfully	Pass

12.4.7 Testing Techniques Used

- **Equivalence Partitioning:** Inputs divided into valid and invalid groups.
- **Error Guessing:** Based on possible user mistakes and edge cases.

12.4.8 Conclusion

The system works correctly for both valid and invalid inputs. All modules behave as expected.

Functions Covered

Controllers:

- onNewClassAdd
- leavingClass

Database:

- studentDatabase
- onNewClassAddDatabase
- leavingClassDatabase

A. onNewClassAdd()

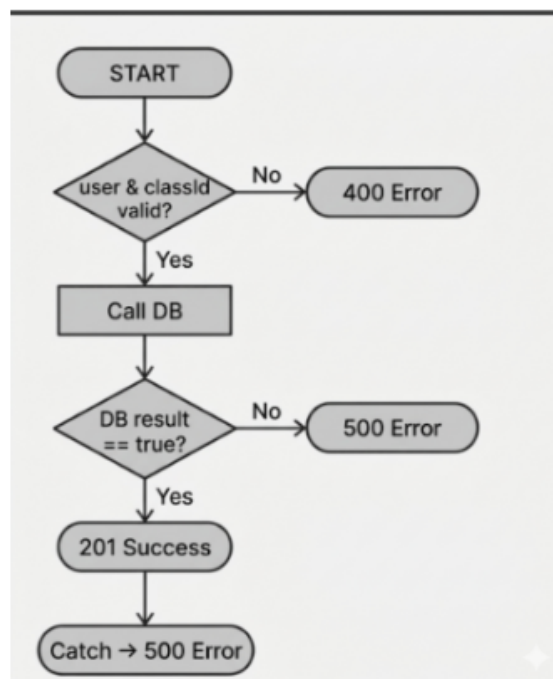


Figure 6: Image 3

Cyclomatic Complexity

- Decision Points:
 - `if (!req.user || !classId)`
 - `if (result == true)`
 - `catch`
- $M = 3 + 1 = 4$

Test Cases**Route:** POST /newclass-enroll**LX11**

TC	Input	Expected Output	Path
TC5	No user	400 error	Validation fail
TC6	No classId	400 error	Validation fail
TC7	Valid + DB true	201 success	Success
TC8	Valid + DB false	500 error	Failure
TC9	Exception	500 error	Catch

TC5: Missing classId**Input:**

```
{}
```

Expected:

```
{
  "error": "Missing required fields"
}
```

TC6: Valid Request**Input:**

```
{
  "classId": "class101"
}
```

Headers:

```
{
  "Authorization": "Bearer VALID_TOKEN"
}
```

Expected:

```
{
  "data": true,
  "message": "Added to new class successfully"
}
```

TC7: Class Not Exist**Input:**

```
{
  "classId": "invalidClass"
}
```

Expected:

```
{
  "error": "Process couldn't be done completely due to some reason"
}
```

B. leavingClass()

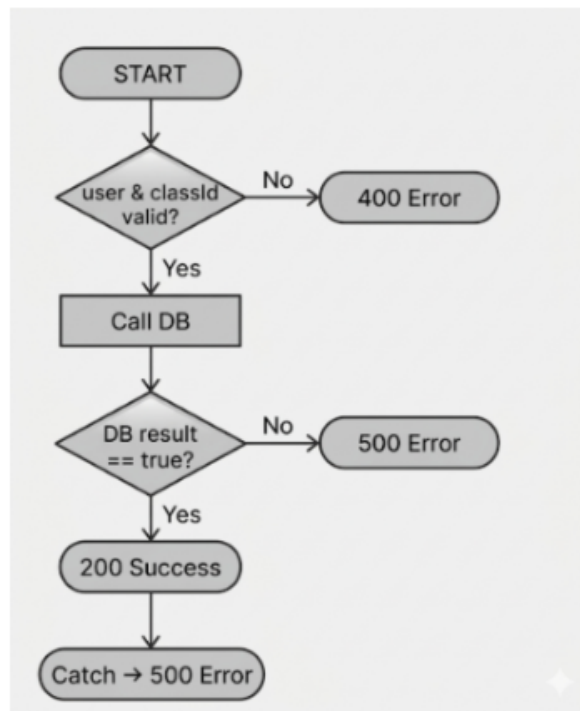


Figure 7: Image 4

Cyclomatic Complexity

- Decision Points:
 - if (!user || !classId)
 - if (result == true)
 - catch
- $M = 3 + 1 = 4$

Test Cases

LX11

TC	Input	Expected Output	Path
----	-------	-----------------	------

TC10	Missing user	400 error	Validation
------	--------------	-----------	------------

TC11	Missing classId	400 error	Validation
------	-----------------	-----------	------------

TC12 Valid + DB true 200 success Success

TC13 Valid + DB false 500 error Failure

TC14 Exception 500 error Catch

TC10: Missing user**Input:**

```
{
  "classId": "class101"
}
```

Expected:

```
{
  "error": "Missing required fields"
}
```

TC11: Valid Request**Input:**

```
{
  "user": {
    "uid": "user123"
  },
  "classId": "class101"
}
```

Expected:

```
{
  "data": true,
  "message": "Left the class successfully"
}
```

TC12: DB Failure (Simulated)**Input:**

```
{
  "user": {
    "uid": "invalid"
  },
  "classId": "class101"
}
```

Expected:

```
{
  "error": "Process couldn't be done completely due to some reason"
}
```

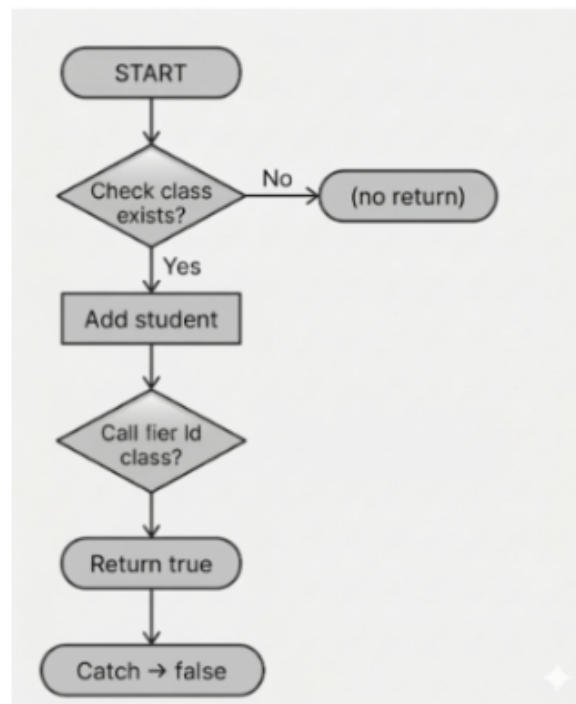



Figure 8: Image 5

C. onNewClassAddDatabase()

Cyclomatic Complexity

- Decision Points:
 - `if (snapshot.exists())`
 - `catch`
- $M = 2 + 1 = 3$

Test Cases

LX1

TC Condition Expected

TC15 Class exists true

TC16 Class not exists undefined

TC17 DB error false

D. leavingClassDatabase()

Cyclomatic Complexity

- Decision Points:

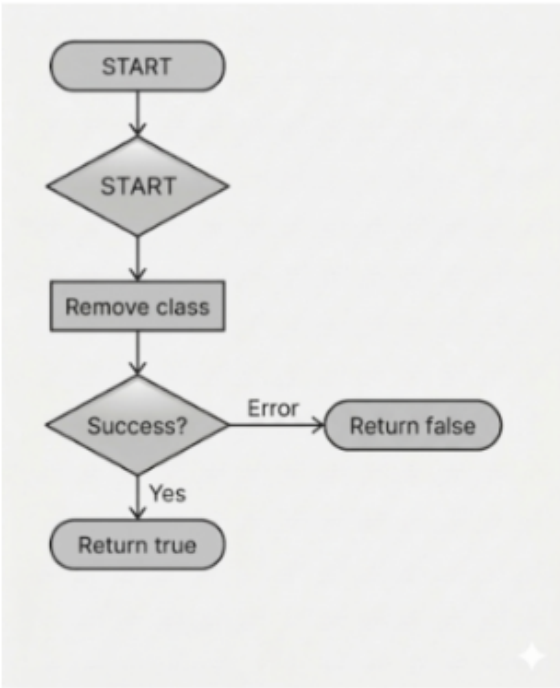


Figure 9: Image 6

- catch
- $M = 1 + 1 = 2$

Test Cases

LXl		
TC	Condition	Expected
TC18	Valid	true
TC19	DB error	false

Conclusion and Future Work

Conclusion

The Student Attendance Management System successfully provides a digital solution to manage and monitor student attendance efficiently. The system replaces traditional manual methods with an automated, secure, and user-friendly web-based platform. It enables faculty to mark and manage attendance easily, students to view their attendance records transparently, and administrators to oversee the entire system effectively.

The implementation of role-based access control ensures data security and prevents unauthorized access. The use of cloud-based storage allows real-time data synchronization and

scalability. Overall, the system improves accuracy, reduces manual errors, saves time, and enhances the overall efficiency of attendance management in educational institutions.

Future Work

Although the current system meets the basic requirements, several enhancements can be implemented in the future to improve its functionality and usability:

- Integration of biometric or face recognition systems for automated attendance marking.
- Development of a mobile application for easier access on smartphones.
- Implementation of notification systems (email/SMS) to alert students about attendance shortages.
- Advanced analytics and visualization dashboards for better insights into attendance patterns.
- Integration with academic management systems for seamless data sharing.
- Addition of offline mode support to handle poor internet connectivity.
- Enhanced security features such as multi-factor authentication.

These future improvements will make the system more robust, intelligent, and suitable for large-scale institutional use.